

PRUEBA TÉCNICA

Desarrollador Fullstack

Aplicaciones Contables

Stack: FastAPI · Next.js/React · PostgreSQL

 **5 días calendario**

Confidencial — Uso interno de selección

1. Contexto

Nuestra plataforma desarrolla soluciones contables para empresas colombianas. Como parte de este proceso, construirás un módulo que permita registrar operaciones contables, consultar sus efectos sobre las cuentas y generar información estructurada a partir de los datos registrados.

No buscamos la implementación de un ERP completo. Buscamos entender cómo modelas el dominio contable, cómo proteges la integridad de la información y cómo construyes una solución fullstack mantenible.

 Antes de empezar, lee esto
No es obligatorio implementar todo el alcance descrito.
Esperamos que tomes decisiones de alcance conscientes dentro del tiempo disponible.
Si decides no implementar alguna funcionalidad, documenta brevemente la decisión, el impacto estimado y cómo la abordarías en una siguiente iteración.
Se valorará especialmente: la consistencia del dominio, la calidad de las decisiones técnicas, la integridad de los datos y la capacidad de entregar un flujo funcional de extremo a extremo.

2. Tecnologías requeridas

El stack es fijo. No está permitido reemplazar ninguna capa principal.

Stack obligatorio
Backend: FastAPI · Python
Base de datos: PostgreSQL
Frontend: Next.js/React

3. Reto principal — Motor contable

La aplicación debe permitir administrar un plan de cuentas y registrar comprobantes contables.

3.1 Plan de cuentas

El sistema debe poder gestionar un plan de cuentas básico. Cada cuenta tiene al menos: un código, un nombre y una naturaleza (débito o crédito). Las cuentas pueden estar activas o inactivas.

La estructura interna del plan de cuentas (jerarquía, niveles, agrupación) queda a tu criterio de diseño. Explica brevemente la decisión.

3.2 Comprobante contable

Un comprobante contiene:

- Empresa.
- Fecha y período contable.
- Descripción.
- Estado (al menos: borrador y contabilizado).
- Una o más líneas contables, donde cada línea tiene: cuenta, tercero cuando aplique, débito, crédito y descripción opcional.

3.3 Reglas del sistema

Antes de contabilizar un comprobante, el sistema debe garantizar:

1. Que tenga al menos dos líneas.
2. Que los valores sean válidos (positivos, sin decimales excesivos).
3. Que ninguna línea tenga simultáneamente débito y crédito.
4. Que el total de débitos sea igual al total de créditos (partida doble).
5. Que todas las cuentas utilizadas estén activas.
6. Que el período contable esté abierto.

Cuando un comprobante es contabilizado:

7. Debe quedar protegido frente a modificaciones que alteren su información contable.
8. Sus movimientos deben reflejarse correctamente en las consultas del libro mayor.
9. La operación debe ser consistente ante errores parciales.

3.4 Reversión de comprobantes

Un comprobante contabilizado puede requerir reversión. Define e implementa una estrategia contablemente consistente.

Hay más de una forma válida de hacerlo. Lo que importa es que la solución mantenga la trazabilidad y la integridad del libro mayor.

3.5 Cierre de período

La aplicación debe permitir cerrar un período contable. Después del cierre no deben poder registrarse nuevos comprobantes que afecten ese período.

En el README documenta cómo resolverías una reapertura del período. No es obligatorio implementarla.

4. Escenarios que la solución debe manejar

Tu implementación debe comportarse correctamente en los siguientes casos. Úsalos como guía de validación, no como lista de tareas mecánicas.

Escenario 1 — Comprobante válido (una compra)

Cuenta	Naturaleza	Débito	Crédito
Gasto operacional	Débito	1.000.000	—
IVA descontable	Débito	190.000	—
Proveedores	Crédito	—	1.190.000

El sistema debe permitir contabilizar este comprobante.

Escenario 2 — Comprobante desbalanceado

Cuenta	Naturaleza	Débito	Crédito
Caja	Débito	500.000	—
Ingresos	Crédito	—	450.000

El sistema debe rechazar la contabilización con un mensaje claro que indique la causa.

Escenario 3 — Reversión

Un usuario detecta que un comprobante ya contabilizado contiene un error. La aplicación debe permitir corregir la situación conservando la trazabilidad completa de lo ocurrido.

Escenario 4 — Período cerrado

El período 2025-01 fue cerrado. Un usuario intenta contabilizar un comprobante fechado en enero de 2025. El sistema debe rechazar la operación indicando el motivo.

Escenario 5 — Precisión monetaria

Los valores monetarios no deben sufrir errores de representación. La solución debe manejar correctamente la precisión numérica a lo largo de toda la pila tecnológica.

Escenario 6 — Operaciones concurrentes

Dos solicitudes intentan contabilizar comprobantes simultáneamente sobre la misma empresa y período. La solución no debe producir estados contables inconsistentes ni numeraciones duplicadas.

No es necesario resolver esto con una solución de alta complejidad. Si es necesario abordarlo: documenta cómo lo resuelves y qué garantías ofrece tu implementación.

5. Libro mayor

Implementa una consulta de libro mayor que permita seleccionar una cuenta y un rango de fechas, y visualizar:

- Fecha del comprobante.
- Número o referencia del comprobante.
- Descripción.
- Tercero, cuando aplique.
- Débito del movimiento.
- Crédito del movimiento.
- Saldo acumulado.

La estrategia para calcular el saldo acumulado queda a tu criterio. Puede ser cálculo en tiempo real desde movimientos, mantenimiento de acumulados, o una combinación. Explica en el README la decisión y sus implicaciones en términos de consistencia, rendimiento y concurrencia.

6. Información exógena simplificada

A partir de los movimientos contables asociados a terceros, implementa la generación de un archivo XML de información exógena simplificada para un año gravable.

6.1 Estructura del archivo

Estructura XML requerida
<InformacionExogena version="1.0">
<Informante nit="..." dv="..." razonSocial="..." anioGravable="..." />
<Registros>
<Registro tipoDoc="..." numDoc="..." nombre="..." concepto="..."
valorBruto="..." valorRetencion="..." />
<!-- un Registro por cada tercero/concepto -->

</Registros>
<Totales registros="..." totalValorBruto="..." totalRetencion="..." />
</InformacionExogena>

6.2 Reglas de negocio

10. El NIT del informante debe validarse con el algoritmo del dígito de verificación antes de incluirse en el archivo.
11. Los movimientos deben agruparse por tercero y concepto.
12. El sistema debe soportar un umbral mínimo expresado en UVT. Para determinar si un tercero debe incluirse, convierte el umbral a pesos usando el valor de UVT correspondiente al año gravable. Si el valor total de un tercero no supera el umbral, excluirlo del reporte con trazabilidad en el log.
13. Los totales de control deben cuadrar con la suma de los registros incluidos.
14. Cada generación debe quedar registrada con fecha, parámetros usados y un identificador que permita re-descargar el archivo posteriormente.

6.3 Endpoints mínimos

- POST /api/exogena/generar — recibe parámetros, genera el XML y lo retorna como descarga directa.
- GET /api/exogena/historial — listado de generaciones previas.
- GET /api/exogena/historial/{id}/archivo — re-descarga de una generación previa.

7. Integración externa — Valor de la UVT

La aplicación necesita conocer el valor de la UVT por año para aplicar umbrales en la generación de exógena. Implementa una estrategia para obtener y mantener actualizado este valor desde una fuente externa, sin bloquear una petición HTTP en curso.

La fuente puede ser:

- Una API pública que retorne el valor de la UVT.
- Una página pública con este dato.
- Un proveedor simulado que represente una integración externa real.

La solución debe considerar:

- Fallos temporales de la fuente externa.
- Reintentos ante fallas transitorias.
- Que ejecuciones repetidas no generen duplicados.
- Trazabilidad del resultado de cada ejecución.

Si el sitio elegido bloquea el acceso durante la prueba, documenta el intento y usa un proveedor simulado. El código de integración importa más que los datos reales.

8. Frontend

Construye las siguientes vistas en Next.js con TypeScript. Usa Server Components donde aplique y Client Components donde haya interacción del usuario.

Vista 1 — Registro de comprobantes

Debe permitir:

- Agregar y eliminar líneas contables.
- Seleccionar cuenta para cada línea.
- Ingresar tercero cuando aplique.
- Visualizar el total de débitos, total de créditos y su diferencia en tiempo real.
- Guardar como borrador y contabilizar.
- Manejar y mostrar errores de validación del backend.

Vista 2 — Libro mayor

Debe permitir:

- Filtrar por cuenta y rango de fechas.
- Visualizar los movimientos con saldo acumulado.
- Mostrar estados de carga, estado vacío y errores.

Vista 3 — Generación de información exógena

Debe permitir:

- Configurar los parámetros de generación (empresa, año gravable, umbral).
- Descargar el XML generado.
- Consultar el historial de generaciones previas y re-descargar.

9. Arquitectura y calidad

9.1 Separación de responsabilidades

La solución debe mostrar una separación clara entre las responsabilidades de negocio, persistencia, exposición de API y presentación. La estructura concreta queda a tu criterio y debe ser explicada brevemente en el README.

9.2 Pruebas automatizadas

Incluye pruebas automatizadas que consideres necesarias para proteger las reglas de negocio de mayor riesgo.

No esperamos cobertura exhaustiva de cada línea de código. Nos interesa entender qué decidiste probar y por qué.

9.3 Consideraciones de dominio

A lo largo de la implementación, ten en cuenta:

- Los valores monetarios no deben sufrir pérdida de precisión por representación en punto flotante. Elige el tipo adecuado en cada capa.
- Las operaciones que modifican el estado contable deben ser consistentes ante fallos parciales.
- Las operaciones relevantes del ciclo de vida de un comprobante deben ser auditables.

10. README

El README es parte de la evaluación. Debe incluir:

15. Instrucciones para levantar el proyecto en local.
16. Cómo ejecutar las migraciones.
17. Cómo correr las pruebas.
18. Decisiones de diseño relevantes: qué modelaste así y por qué.
19. Limitaciones conocidas de la entrega.
20. Pendientes: qué faltó y cómo lo abordarías.
21. "¿Qué cambiarías para llevar esta solución a producción?" — responde esto aunque sea brevemente.

11. Extensión opcional

Si el núcleo de la solución está completo, puedes implementar una extensión que consideres valiosa. Algunas ideas:

- Auditoría avanzada del ciclo de vida de los comprobantes.
- Exportación del libro mayor a Excel.
- Importación masiva de comprobantes desde CSV.
- Autenticación básica con JWT.
- Docker Compose que levante todo el sistema con un comando.
- Pipeline de CI básico en GitHub Actions.
- Gráfica de evolución de saldos por cuenta.
- Cualquier otra mejora que consideres valiosa y que puedas justificar.

No es un bonus con puntaje. Es una señal de criterio: qué priorizas cuando tienes tiempo libre y por qué.

En el README explica qué elegiste y por qué.

12. Entrega

Instrucciones	
	Repositorio público en GitHub o GitLab.
	Envía el enlace al correo indicado por el reclutador.



Plazo: 5 días calendario desde la recepción de este documento.



Si no completaste un módulo, documenta el avance y lo que falta — eso también se evalúa.

El evaluador intentará levantar tu proyecto siguiendo únicamente el README. Asegúrate de que las instrucciones sean completas y funcionen en un entorno limpio.

Después de la entrega habrá una sesión técnica de 45–60 minutos donde revisaremos el código juntos. Prepárate para explicar tus decisiones, los trade-offs que evaluaste y cómo modificarías partes del sistema ante nuevos requerimientos.

¡Mucho éxito.

Nos interesa entender cómo piensas, no solo qué construyes.

Confidencial · Uso exclusivo para proceso de selección